BAD702 – Statistical Machine Learning for Data Science

Module 4: Regression and Prediction • VTU Dec 2025/Jan 2026 paper • Answers based only on the textbook

Q.7 a. Explain factor (categorical) variables in regression. How are they handled in statistical modeling? [10 Marks]

1. Introduction

Factor variables, also termed **categorical variables**, take on a limited number of discrete values. Example: a loan purpose can be “debt consolidation”, “wedding”, “car”; property type can be Multiplex, Single Family or Townhouse. A **binary (yes/no) variable**, also called an **indicator variable**, is a special case. Regression requires **numerical inputs**, so factor variables must be recoded before use. The most common approach is to convert the variable into a set of **binary dummy variables**.

Key term	Meaning
Dummy variables	Binary 0–1 variables derived by recoding factor data for use in regression and other models.
Reference coding (treatment coding)	Most common coding for statisticians: one level is the reference and the other levels are compared to it.
One hot encoder	Common in machine learning: all factor levels are retained; not appropriate for multiple linear regression.
Deviation coding (sum contrasts)	Compares each level against the overall mean instead of the reference level.

2. Dummy variable representation

The factor PropertyType has three levels, so we create a binary variable for each value (R: `model.matrix(~PropertyType -1, data=house)`; Python: `pd.get_dummies(house['PropertyType'])`). This representation with all levels is called **one hot encoding**. It is the standard way to represent factors in machine learning methods such as nearest neighbors and tree models.

Record	PropertyType Multiplex	Single Family	Townhouse
1 (Multiplex)	1	0	0
2 (Single Family)	0	1	0
6 (Townhouse)	0	0	1

3. Handling in regression – P − 1 dummies

- In regression a factor with **P distinct levels** is usually represented by a matrix with only **P − 1 columns**.
- Reason: a regression model typically includes an **intercept**. Once the P − 1 binaries are defined, the value of the P-th is known and is redundant. Adding the P-th column causes a **multicollinearity error** (perfect redundancy among predictors).
- In Python use `pd.get_dummies(..., drop_first=True)` to get P − 1 columns. In R the default is to produce P − 1 columns using the first factor level as the reference.

4. Reference coding and interpretation

R uses the **first level as the reference** and interprets the remaining levels relative to it. For the house-price model (`lm(AdjSalePrice ~ SqFtTotLiving + SqFtLot + Bathrooms + Bedrooms + BldgGrade + PropertyType)`) the output has PropertyTypeSingle Family = −8.468e+04 and PropertyTypeTownhouse = −1.151e+05. There is no coefficient for **Multiplex** – it is implicitly defined when both dummies are 0.

- The coefficients are interpreted **relative to Multiplex**: a Single Family home is worth about $85,000 less, and a Townhouse about $115,000 less, other variables held constant.
- The result looks unintuitive, but is explained by **location as a confounding variable** (an important omitted predictor).

5. Factor variables with many levels

- Some factors create a huge number of dummies – zip codes: there are 43,000 in the US and 80 in King County. Including all levels needs 79 coefficients (79 degrees of freedom) against only 5 in the original model, and several zip codes have only one sale.
- Solution: explore the data and **consolidate levels**. Options: use the first 2–3 digits of the zip code; group by another variable such as sale price; best, **form groups using the residuals of an initial model** – e.g. compute the median residual per zip code and split the sorted zip codes into five groups (ZipGroup) using `ntile` / `pd.qcut`.

6. Ordered factor variables

Some factors reflect ordered levels (e.g. loan grade A, B, C; BldgGrade 1 = Cabin … 13 = Mansion). These **ordered factors** can often be converted to numeric values and used as is. Treating them as numeric **preserves the information in the ordering** that would be lost if converted to an unordered factor. **Polynomial coding** is a contrast coding suitable for ordered factors.

7. Other points

- Multicollinearity is not a serious problem for nonlinear methods such as trees, clustering and nearest neighbors; there it may be advisable to retain all P dummies.
- Apart from ordered factors, data scientists generally meet only reference coding or one hot encoding.

Key ideas / Conclusion

- Factor variables must be converted into numeric variables for regression; the common method uses P − 1 dummy variables with a reference level.
- Factors with many levels may need consolidation; ordered factors can be treated as a single numeric variable.

Q.7 b. Describe key regression diagnostics techniques. Explain how they help to improve model quality? [10 Marks]

1. Introduction

In explanatory modeling, various steps beyond metrics such as RMSE and R-squared are taken to assess how well the model fits the data. Most are based on **analysis of the residuals**. They do not directly address predictive accuracy, but they give useful insight in a predictive setting.

2. Outliers and standardized residuals

- An **outlier** is a record whose actual y value is distant from the predicted value (a large residual).
- Detected using the **standardized residual** = residual ÷ standard error of the residuals; it means "the number of standard errors away from the regression line". There is no statistical theory separating outliers from non-outliers – only rules of thumb (e.g. beyond ±2.5 on the influence plot).
- **Example (zip 98105):** the smallest standardized residual is −4.33, an overestimate of $757,754. The record was a sale of only partial interest in the property – anomalous – so it should be **removed** from the regression. Outliers may also come from data-entry ("fat-finger") errors or unit mismatches.
- In big data, outliers hardly matter for prediction, but they are central to **anomaly detection** (e.g. fraud).

3. Influential values, leverage, hat-value and Cook's distance

- An **influential value** is one whose absence would significantly change the regression equation; it need not have a large residual. **Leverage** is the degree of influence of a single record.
- **Hat-value:** a common measure of leverage; values above $2(P + 1)/n$ indicate high leverage.
- **Cook's distance:** combines leverage and residual size; high influence if it exceeds $4/(n - P - 1)$.
- **Influence (bubble) plot:** plots hat-values (x), standardized residuals (y) and Cook's distance (bubble size) together.
- In the example, removing influential points (Cook's distance > 0.08) changed the Bathrooms coefficient from 2,282 to −16,132 – a dramatic change. Influence checks matter mainly in smaller data sets and for anomaly detection.

4. Heteroskedasticity, non-normality and correlated errors

- **Heteroskedasticity:** lack of constant residual variance across the range of predicted values. Diagnosed by plotting **absolute residuals vs predicted values** with a smoother (loess). In the example the variance rises for higher-valued homes, so the model is heteroskedastic – it may indicate a **missing predictor**.
- **Non-normal residuals:** checked with a histogram of standardized residuals (example: longer tails than normal and mild skewness). Normal errors suggest a complete model; otherwise the model may be missing something. This mainly affects formal inference (p-values, confidence intervals), which matters little to data scientists.
- **Correlated errors:** for data over time or space, the **Durbin-Watson statistic** detects autocorrelation; correlated errors can be built into the model for short-term forecasts.

5. Partial residual plots and nonlinearity

$$\textbf{Partial residual} = \textbf{Residual} + b_i X_i$$

A **partial residual plot** (added variables plot) isolates the relationship between predictor X_i and the response, taking account of all other predictors. It plots X_i on the x-axis and the partial residual (a "synthetic outcome") on the y-axis, usually with a smooth. For SqFtTotLiving the relationship is evidently **nonlinear**: the linear fit underestimates prices below 1,000 sq ft and overestimates between 2,000 and 3,000 sq ft. R: `predict(lm, type='terms')`; Python: `sm.graphics.plot_ccpr`.

6. How diagnostics improve model quality

Diagnostic	What it reveals	Improvement
Standardized residual	Outliers, bad or anomalous records	Remove/correct the records; investigate anomalies
Hat-value, Cook's distance	Highly influential records	Check/remove so coefficients are stable
Residual vs predicted plot	Heteroskedasticity – something unaccounted for	Add missing predictors; complete the model
Residual histogram, Durbin-Watson	Non-normal or correlated errors	Valid confidence intervals; model autocorrelation
Partial residual plot	Nonlinear relationship with a predictor	Add polynomial / spline / GAM terms
Residuals by group	Information hidden in factor levels	Group zip codes by median residual (ZipGroup)

Conclusion

Diagnostics based on residuals help detect bad records, influential points, a missing variable and nonlinearity, leading to a cleaner and more complete model. For data scientists, predictive accuracy is the main aim, so satisfying distributional assumptions only to validate p-values is not that important.

Q.8 a. How do you interpret a regression equation with both continuous and factor variables with example? [10 Marks]

1. Introduction

In data science the most important use of regression is prediction, but sometimes insight into the relationship between predictors and outcome is needed from the equation itself. Interpretation needs care when the model has **continuous predictors** (e.g. SqFtTotLiving, Bathrooms) and **factor predictors** (e.g. PropertyType, ZipGroup).

2. Example – King County house price model

```
lm(AdjSalePrice ~ SqFtTotLiving + SqFtLot + Bathrooms + Bedrooms + BldgGrade
+ PropertyType + ZipGroup)
```

Variable	Type	Coefficient
(Intercept)	–	−6.666e+05
SqFtTotLiving	continuous	210.6
SqFtLot	continuous	0.455
Bathrooms	continuous	5,928
Bedrooms	continuous	−41,680
BldgGrade	ordered, numeric	98,540
PropertyType Single Family / Townhouse	factor (reference = Multiplex)	+19,320 / −78,200
ZipGroup 2, 3, 4, 5	factor (reference = ZipGroup 1)	+53,320, +116,300, +178,400, +338,400

3. Rules of interpretation

- **Continuous variable:** the coefficient is the change in the outcome for a **one-unit increase** in the predictor, **holding all other variables constant**. SqFtTotLiving = 210.6 → each extra square foot adds about $211 to the predicted price; adding a bathroom increases it by $5,928.
- **Factor variable (reference coding):** each coefficient compares that level with the **reference level**, which has no coefficient (it is the case when all its dummies are 0). A home in ZipGroup 5 is estimated to sell for almost **$340,000 more** than one in ZipGroup 1; a Townhouse is about $78,200 lower than a Multiplex, other things equal.
- **Intercept:** predicted outcome when all continuous predictors are 0 and all factors are at their reference levels.

4. Cautions in interpretation

- **Correlated predictors:** the sign and value of a coefficient are hard to interpret (and standard errors inflate). Bedrooms has a negative coefficient because bigger houses have more bedrooms and size drives value; for the same size, more but smaller bedrooms is less desirable. When SqFtTotLiving, SqFtFinBasement and Bathrooms were removed, the Bedrooms coefficient became positive (acting as a proxy for size).
- **Multicollinearity:** perfect redundancy (e.g. P dummies instead of P − 1, a variable included twice, two nearly perfectly correlated variables) makes the regression unstable or impossible to solve; variables must be removed.

- **Confounding variables:** an important predictor *omitted* from the model gives spurious relationships. Before adding location (ZipGroup), SqFtLot and Bathrooms had negative coefficients; after adding it they became positive.

5. Main effects and interactions

Main effects are the ordinary predictors; using only main effects assumes that the relationship between a predictor and the response is independent of the other predictors. This is often untrue. An **interaction** (R: `SqFtTotLiving * ZipGroup`) is needed when the effect of one variable depends on the level of another.

- For the lowest ZipGroup the slope for SqFtTotLiving is the main effect, about $115 per sq ft.
- For the highest ZipGroup the slope is the main effect plus the interaction: $115 + $227 = **$342 per sq ft**, almost three times as much.
- Interaction terms can be chosen using prior knowledge, stepwise selection, penalized regression, or tree models.

Conclusion

Interpret continuous coefficients as per-unit changes and factor coefficients as differences from the reference level, always holding other variables constant, while watching for correlated predictors, multicollinearity, confounding variables and interactions.

Q.8 b. Explain Spline Regression. How is it different from polynomial Regression? Describe its advantages. [10 Marks]

1. Introduction – nonlinear relationships

The relationship between the response and a predictor is not necessarily linear. The response to a drug dose is often nonlinear (doubling the dose does not double the response), and demand is not linear in marketing spend because it saturates. Regression can be extended to capture such effects by **polynomial regression**, **spline regression** and **generalized additive models (GAM)**.

Key term	Meaning
Polynomial regression	Adds polynomial terms (squares, cubes, etc.) to a regression.
Spline regression	Fitting a smooth curve with a series of polynomial segments.
Knots	Values that separate spline segments.
GAM	Spline models with automated selection of knots.

2. Polynomial regression

Polynomial terms are included in the equation, e.g. the quadratic regression:

$$Y = b_0 + b_1X + b_2X^2 + e$$

In R: `poly(SqFtTotLiving, 2)`; in statsmodels: `I(SqFtTotLiving**2)`. There are two coefficients for the predictor – a linear and a quadratic term. It captures only a certain amount of curvature; adding higher-order terms (cubic, quartic) often leads to undesirable **“wiggliness”**.

3. Spline regression

- A **spline** is technically a **series of piecewise continuous polynomials**, smoothly connected at fixed points of the predictor called **knots**. Splines originally were thin bendable wood strips held by weights (“ducks”) used by draftsmen to draw smooth curves in ship and aircraft building; the mathematical splines were developed in World War II by I. J. Schoenberg.
- Two parameters must be specified: the **degree** of the polynomial and the **location of the knots**.
- R: `bs(SqFtTotLiving, knots=knots, degree=3)` with knots at the 25th, 50th, 75th percentiles (a cubic b-spline). statsmodels: `bs(SqFtTotLiving, df=6, degree=3)` gives df − degree = 3 internal knots.
- Unlike a linear term, spline coefficients are **not interpretable**; use the visual (partial residual) display to see the fit.
- **GAM** automates knot selection: `gam(AdjSalePrice ~ s(SqFtTotLiving) + …)` in R; `LinearGAM(s(0, n_splines=12) + l(1) …)` in Python (pyGAM; the default 20 splines overfits at large sizes).

4. Difference between polynomial and spline regression

Point	Polynomial regression	Spline regression
Form	One polynomial over the whole range of X	Series of polynomial segments joined at knots

Flexibility	Limited curvature	Greater flexibility; follows the smooth more closely
Higher order	Cubic/quartic terms cause wiggliness	Each segment is low-degree, so smooth without wiggles
Specification	Choose the degree only	Choose degree and knots (or let GAM choose)
Coefficients	Linear and quadratic terms are reported	Coefficients not interpretable; use plots
Complexity	Simple to set up	Formulation more complicated; software handles it

5. Advantages of spline regression

- Models nonlinear relationships more flexibly and **smoothly** than polynomials; in the housing example the spline fit matched the smooth of the partial residuals more closely than the quadratic fit.
- Avoids the **wiggliness** of high-degree polynomials by using low-degree pieces.
- Knots can be placed where the data change shape (e.g. at quartiles), and **GAM** selects them automatically.
- It is still fitted as a regression model within the usual least-squares framework; nonlinear regression in the strict sense is harder and computationally heavier, so linear models are preferred where possible.
- Extends to other models: splines and GAMs can be used in logistic regression too.

Caution / Conclusion

A better-fitting spline is not automatically a better model – in the example it implied that very small homes (< 1,000 sq ft) are worth more than slightly larger ones, which makes no economic sense and is probably the effect of a confounding variable. Spline regression joins low-degree polynomial pieces at knots to give a smooth, flexible fit for nonlinear relationships, avoiding the wiggliness of high-order polynomials.